SOCIETY MANAGEMENT PLATFORM

Phase 1 – Society Core Database Design

Internal Guide: Saraswati Patil

Team: Anushree Kendale • Om Karkele • Shreya Khatavkar • Chinmay Khiste • Madhur Biradar

FINAL – Revised after Industry Review

1. Scope and Final Modeling Update

This document defines the revised Phase 1 Society Core database design after the latest industry review. Phase 1 covers society and property master data, centralized authentication identity, society membership and RBAC, person records, residents, family members, emergency contacts, ownership, leases, occupancy history and vehicles. Security, maintenance, facilities, helpdesk, governance, accounting, documents and SaaS modules remain in their respective later phases.

The main structural revision is the introduction of a centralized persons table. In the earlier design, owner, resident and tenant records repeated personal fields such as name, mobile and email. This could cause inconsistent personal data for the same individual and mixed responsibility between identity and relationship records. The revised model stores a person's common details once in persons, while residents, ownerships and leases represent the person's relationship with a society or unit.

users remains the authentication/account table. A user account may be linked to one person, while society_memberships and role mappings determine access within each society. This keeps authentication, person identity and business relationships separate.

2. Finalized Modeling Principles

- Use persons as the single source for common real-world personal information.
- Use users for authenticated application accounts; do not duplicate person details in role-specific business tables.
- Use society_memberships to support one user belonging to multiple societies with different access.
- Assign roles through society membership, not directly through the global users record.
- Keep roles and permissions separate so authorization can evolve independently from identity.
- Use a shared PostgreSQL database with society_id tenant isolation for society-owned records.
- Derive society context from authenticated membership; clients must not select an arbitrary society_id.
- Use UUID/ULID-style identifiers for externally exposed records.
- Preserve ownership, lease and occupancy history rather than overwriting historical relationships.
- Use societies → buildings → floors → units as the standard property hierarchy; building_type can distinguish TOWER, BLOCK, WING, etc.
- Keep security-specific tables out of Phase 1; gates, visitors, gate_events, deliveries, helpers and incidents belong to Phase 2.

3. Phase 0 Dependencies Included for Phase 1

The following identity and authorization tables are shown because Phase 1 business records depend on them. They are implemented as the Phase 0 foundation before Society Core workflows.

3.1. societies

Phase: Phase 0–1 | Purpose: Tenant master representing one residential society/community.

Field	Type	Null	Key	Description
society_id	UUID	NO	PK	Unique society identifier
name	VARCHAR(150)	NO	UNI	Society name
registration_no	VARCHAR(100)	YES	UNI	Society registration number
address	TEXT	YES	—	Society address
contact_phone	VARCHAR(20)	YES	—	Primary contact number
contact_email	VARCHAR(150)	YES	—	Primary contact email
logo_path	VARCHAR(500)	YES	—	Private object-storage path
status	VARCHAR(20)	NO	IDX	Society lifecycle status
created_at	TIMESTAMPTZ	NO	IDX	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Society is the tenant boundary for society-owned data.

3.2. society_settings

Phase: Phase 0–1 | Purpose: Stores configurable society-level operational and billing settings.

Field	Type	Null	Key	Description
setting_id	UUID	NO	PK	Settings identifier
society_id	UUID	NO	FK, UNI	Owning society
financial_year_start	DATE	YES	—	Financial year start
billing_cycle	VARCHAR(20)	YES	—	Maintenance billing cycle
late_fee_rule	JSONB	YES	—	Configured late-fee rule
grace_period_days	SMALLINT	YES	—	Configured grace period
due_day	SMALLINT	YES	—	Default bill due day
gate_timing	JSONB	YES	—	Gate timing configuration
visitor_policy	JSONB	YES	—	Visitor policy configuration
emergency_config	JSONB	YES	—	Emergency configuration
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: One settings record per society.

3.3. users

Phase: Phase 0 | Purpose: Authentication/account record for platform users.

Field	Type	Null	Key	Description
user_id	UUID	NO	PK	Unique user identifier
person_id	UUID	NO	FK, UNI	Linked real-world person
password_hash	VARCHAR(255)	YES	—	Password hash where password login is enabled

is_active	BOOLEAN	NO	—	Account status
last_login_at	TIMESTAMPTZ	YES	—	Last successful login
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: No role column belongs here; role is determined through membership and role mapping. Person details are maintained in persons.

3.4. society_memberships

Phase: Phase 0 | Purpose: Associates a user with a society and establishes tenant context.

Field	Type	Null	Key	Description
membership_id	UUID	NO	PK	Membership identifier
society_id	UUID	NO	FK,IDX	Society
user_id	UUID	NO	FK,IDX	User account
status	VARCHAR(20)	NO	IDX	Membership status
joined_at	TIMESTAMPTZ	NO	—	Membership start time
left_at	TIMESTAMPTZ	YES	—	Membership end time if applicable
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Unique constraint on (society_id, user_id) prevents duplicate membership records.

3.5. roles

Phase: Phase 0 | Purpose: Defines reusable platform roles.

Field	Type	Null	Key	Description
role_id	UUID	NO	PK	Role identifier
name	VARCHAR(80)	NO	UNI	Role name
description	VARCHAR(255)	YES	—	Role description
is_active	BOOLEAN	NO	—	Whether role is active
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Examples include Society Admin, Owner, Resident, Tenant and other approved roles; exact catalogue is finalized during implementation.

3.6. permissions

Phase: Phase 0 | Purpose: Defines atomic permissions used by RBAC.

Field	Type	Null	Key	Description
permission_id	UUID	NO	PK	Permission identifier
code	VARCHAR(120)	NO	UNI	Permission code
description	VARCHAR(255)	YES	—	Permission description
module	VARCHAR(80)	YES	IDX	Owning module
created_at	TIMESTAMPTZ	NO	—	Creation timestamp

Important constraints: Permissions should be granular enough for server-side authorization.

3.7. role_permissions

Phase: Phase 0 | Purpose: Maps roles to permissions.

Field	Type	Null	Key	Description
role_permission_id	UUID	NO	PK	Mapping identifier
role_id	UUID	NO	FK,IDX	Role
permission_id	UUID	NO	FK,IDX	Permission
created_at	TIMESTAMPTZ	NO	—	Creation timestamp

Important constraints: Unique constraint on (role_id, permission_id).

3.8. user_roles

Phase: Phase 0 | Purpose: Assigns roles to a user's society membership.

Field	Type	Null	Key	Description
user_role_id	UUID	NO	PK	Assignment identifier
membership_id	UUID	NO	FK,IDX	Society membership
role_id	UUID	NO	FK,IDX	Assigned role
valid_from	TIMESTAMPTZ	NO	—	Role validity start
valid_to	TIMESTAMPTZ	YES	—	Role validity end
status	VARCHAR(20)	NO	IDX	Assignment status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp

Important constraints: Role is assigned to membership, allowing different roles for the same user in different societies.

4. Phase 1 Society Core Tables

4.1. buildings

Phase: Phase 1 | Purpose: Stores buildings/towers/blocks/wings belonging to a society.

Field	Type	Null	Key	Description
building_id	UUID	NO	PK	Building identifier
society_id	UUID	NO	FK,IDX	Owning society
name	VARCHAR(100)	NO	—	Building display name
code	VARCHAR(30)	YES	—	Society-specific building code
building_type	VARCHAR(30)	YES	IDX	TOWER, BLOCK, WING or configured type
total_floors	INTEGER	YES	—	Configured number of floors
status	VARCHAR(20)	NO	IDX	Building status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: No separate towers table is required unless a future business distinction is identified.

4.2. floors

Phase: Phase 1 | Purpose: Stores floors within each building.

Field	Type	Null	Key	Description
floor_id	UUID	NO	PK	Floor identifier
building_id	UUID	NO	FK,IDX	Parent building
floor_number	INTEGER	NO	—	Numeric floor number
name	VARCHAR(50)	YES	—	Display name if required
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Unique constraint recommended on (building_id, floor_number).

4.3. unit_types

Phase: Phase 1 | Purpose: Stores classifications of residential units.

Field	Type	Null	Key	Description
unit_type_id	UUID	NO	PK	Unit type identifier
society_id	UUID	NO	FK,IDX	Owning society
name	VARCHAR(50)	NO	—	Unit type name
description	VARCHAR(200)	YES	—	Unit type description
default_area	NUMERIC(12,2)	YES	—	Default area where configured
area_unit	VARCHAR(20)	YES	—	Area unit, e.g. sqft
is_active	BOOLEAN	NO	—	Whether type is active
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Unit types are society-scoped.

4.4. units

Phase: Phase 1 | Purpose: Stores residential flats/units and property master data.

Field	Type	Null	Key	Description
unit_id	UUID	NO	PK	Unit identifier
society_id	UUID	NO	FK,IDX	Owning society
building_id	UUID	NO	FK,IDX	Parent building
floor_id	UUID	NO	FK,IDX	Parent floor
unit_type_id	UUID	YES	FK	Unit classification
unit_number	VARCHAR(30)	NO	—	Flat/unit number
area	NUMERIC(12,2)	YES	—	Unit area
occupancy_status	VARCHAR(20)	NO	IDX	Vacant, occupied or configured status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Unique constraint recommended on (society_id, building_id, unit_number).

4.5. persons

Phase: Phase 1 | Purpose: Central real-world person record used by residents, owners, tenants and other person-linked relationships.

Field	Type	Null	Key	Description
person_id	UUID	NO	PK	Person identifier
full_name	VARCHAR(150)	NO	—	Full legal/display name
mobile	VARCHAR(20)	YES	—	Primary contact mobile
email	VARCHAR(150)	YES	—	Primary email
date_of_birth	DATE	YES	—	Date of birth where required
gender	VARCHAR(30)	YES	—	Gender where required
profile_photo_path	VARCHAR(500)	YES	—	Private object-storage path for profile photo
status	VARCHAR(20)	NO	IDX	Person record status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Common personal data is stored once here. Relationship tables reference person_id instead of duplicating name/mobile/email.

4.6. residents

Phase: Phase 1 | Purpose: Stores approved resident-to-unit relationships.

Field	Type	Null	Key	Description
resident_id	UUID	NO	PK	Resident relationship identifier
society_id	UUID	NO	FK,IDX	Owning society
person_id	UUID	NO	FK,IDX	Linked person
unit_id	UUID	NO	FK,IDX	Current residential unit
resident_type	VARCHAR(30)	NO	—	Owner, tenant, family member or configured resident type
is_primary	BOOLEAN	NO	—	Whether primary resident for the unit where applicable
status	VARCHAR(20)	NO	IDX	Resident status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Resident contact details come from persons. Historical movement is preserved through occupancy_history.

4.7. family_members

Phase: Phase 1 | Purpose: Stores family relationships associated with a resident.

Field	Type	Null	Key	Description
family_member_id	UUID	NO	PK	Family member relationship identifier
society_id	UUID	NO	FK,IDX	Owning society
resident_id	UUID	NO	FK,IDX	Primary resident
person_id	UUID	YES	FK,IDX	Linked person if independently registered
relationship	VARCHAR(50)	NO	—	Relationship to resident
emergency_contact	BOOLEAN	NO	—	Whether marked as emergency contact
app_access_enabled	BOOLEAN	NO	—	Whether independent app access is enabled
status	VARCHAR(20)	NO	IDX	Family member status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: A family member can be linked to a person when independent identity/app access is needed; the person record remains the single source for personal details.

4.8. emergency_contacts

Phase: Phase 1 | Purpose: Stores emergency contacts associated with residents.

Field	Type	Null	Key	Description
emergency_contact_id	UUID	NO	PK	Emergency contact relationship identifier
society_id	UUID	NO	FK,IDX	Owning society
resident_id	UUID	NO	FK,IDX	Resident
person_id	UUID	YES	FK,IDX	Linked person if the contact is registered
name	VARCHAR(150)	YES	—	Contact name when no person record exists
relationship	VARCHAR(50)	YES	—	Relationship to resident
mobile	VARCHAR(20)	YES	—	Emergency contact number
priority	SMALLINT	YES	—	Contact priority
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Emergency contacts remain separate from family_members because an emergency contact may be external to the household. If person_id is present, personal details should be taken from persons.

4.9. ownerships

Phase: Phase 1 | Purpose: Stores person-to-unit ownership relationships and ownership history.

Field	Type	Null	Key	Description
ownership_id	UUID	NO	PK	Ownership record identifier
society_id	UUID	NO	FK,IDX	Owning society

unit_id	UUID	NO	FK,IDX	Owned unit
person_id	UUID	NO	FK,IDX	Owner person
ownership_type	VARCHAR(30)	YES	—	Configured ownership type
share_percentage	NUMERIC(5,2)	YES	—	Ownership share where applicable
start_date	DATE	NO	—	Ownership start date
end_date	DATE	YES	—	Ownership end date
status	VARCHAR(20)	NO	IDX	Ownership status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Do not overwrite ownership history; transfers create new historical ownership records. Multiple owners per unit are supported.

4.10. leases

Phase: Phase 1 | Purpose: Stores tenancy agreements between a tenant person, an owner and a unit.

Field	Type	Null	Key	Description
lease_id	UUID	NO	PK	Lease identifier
society_id	UUID	NO	FK,IDX	Owning society
unit_id	UUID	NO	FK,IDX	Leased unit
tenant_person_id	UUID	NO	FK,IDX	Tenant person
owner_person_id	UUID	YES	FK,IDX	Owner/lessor person where recorded
lease_start	DATE	NO	—	Lease start date
lease_end	DATE	YES	—	Lease end date
status	VARCHAR(20)	NO	IDX	Lease status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Lease history is preserved through separate lease records and dates/status rather than overwriting previous tenancy.

4.11. occupancy_history

Phase: Phase 1 | Purpose: Maintains historical occupancy records for a unit.

Field	Type	Null	Key	Description
occupancy_id	UUID	NO	PK	Occupancy history identifier
society_id	UUID	NO	FK,IDX	Owning society
unit_id	UUID	NO	FK,IDX	Unit
person_id	UUID	NO	FK,IDX	Person occupying the unit
occupancy_type	VARCHAR(20)	NO	—	Owner, tenant, resident or configured type
move_in_date	DATE	NO	—	Move-in date
move_out_date	DATE	YES	—	Move-out date
status	VARCHAR(20)	NO	IDX	Occupancy status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Application logic/database constraints should prevent overlapping active occupancy records for the same unit.

4.12. vehicles

Phase: Phase 1 | Purpose: Stores vehicles linked to people and units.

Field	Type	Null	Key	Description
vehicle_id	UUID	NO	PK	Vehicle identifier
society_id	UUID	NO	FK,IDX	Owning society
person_id	UUID	YES	FK,IDX	Person associated with the vehicle
unit_id	UUID	YES	FK,IDX	Associated unit
registration_number	VARCHAR(30)	NO	—	Vehicle registration number
vehicle_type	VARCHAR(30)	NO	—	Vehicle type
make_model	VARCHAR(100)	YES	—	Make and model
colour	VARCHAR(40)	YES	—	Vehicle colour
status	VARCHAR(20)	NO	IDX	Vehicle status
created_at	TIMESTAMPTZ	NO	—	Creation timestamp
updated_at	TIMESTAMPTZ	NO	—	Last update timestamp

Important constraints: Vehicle registration uniqueness should be enforced within the intended society scope. Vehicle ownership is linked to person rather than resident so owners, tenants or other approved persons can be associated.

5. Phase 1 Relationship Summary

- Society 1:1 Society Settings.
- Society 1:N Society Memberships; User 1:N Society Memberships.
- Society Membership 1:N User Roles; Role 1:N User Roles.
- Role N:M Permission through Role Permissions.
- Society 1:N Buildings; Building 1:N Floors; Floor 1:N Units.
- Society 1:N Unit Types; Unit Type 1:N Units.
- Person 1:0..1 User account; User access is controlled by society membership and RBAC.
- Society 1:N Residents; Person 1:N Resident relationships; Unit 1:N Residents.
- Resident 1:N Family Members; Family Member may optionally reference Person.
- Resident 1:N Emergency Contacts; Emergency Contact may optionally reference Person.
- Person 1:N Ownerships; Unit 1:N Ownerships, creating Person N:M Unit through Ownerships.
- Person 1:N Leases as tenant; Unit 1:N Leases; optional owner-person reference.
- Person 1:N Occupancy History; Unit 1:N Occupancy History.
- Person 1:N Vehicles; Unit 1:N Vehicles.
- Building, floor and unit relationships must be validated against the same society to maintain tenant isolation.

6. Why the Identity Model Was Revised

The earlier Phase 1 model contained owners, residents and tenants as separate person-like records. Each could contain overlapping personal information such as full_name, mobile and email. This was functional for a small prototype but could create duplicate or inconsistent information when the same individual appeared in more than one role.

The revised model separates identity from relationship: persons stores the individual once; users stores the authentication account; residents describes residence; ownerships describes ownership; leases describe tenancy; and occupancy_history preserves historical residence. This reduces redundancy while still allowing one person to have multiple roles or relationships.

7. Phase Boundary

Security-specific tables such as gates, visitors, visitor_passes, visits, gate_events, deliveries, helpers and incidents are intentionally excluded from this Phase 1 database design and will be implemented in Phase 2. Maintenance transactions, payments and receipts are Phase 3; facilities and bookings Phase 4; helpdesk, vendors and assets Phase 5; governance Phase 6; accounting Phase 7; documents and share certificates Phase 8; SaaS subscriptions/plans/feature flags Phase 9; and integration-oriented AI, WhatsApp, OCR, e-sign, IoT/CCTV/intercom work is Phase 10.

8. Implementation Notes

- Authorization is enforced server-side through membership, role and permission checks; hiding UI options is not sufficient.
- Every tenant-owned business record must carry society_id where required, and repository/service queries must apply tenant scope.
- JWT/session context identifies the user; the backend resolves the active society membership instead of trusting a client-supplied society_id.
- Use SQLAlchemy as the ORM/data-access layer and Alembic for database migrations.
- Use PostgreSQL NUMERIC for all financial values in later phases.
- Use object storage metadata and signed URLs for private profile/document files.
- Preserve ownership, lease and occupancy history; normal UI operations must not physically delete historical records.